

README

Helix OTA — Phase 1.0.2 (Operator / User-Initiated Rollback)

Field	Value
Revision	2
Created	2026-06-08
Last modified	2026-06-08
Status	SUPERSEDED — folded into 1.0.1 (operator decision 2026-06-08) End-user / operator rollback is now part of 1.0.1 (the 1.0.1 outline always carried “End-User Rollback” in its title). This dir is retained as a pointer per §11.4.124 (investigate-before-remove), NOT deleted. The rollback mechanics here remain the design reference; their canonical home is 1.0.1-staged-rollout/ . The DB backing — <code>rollback_history</code> — is in migration 002 (<code>1.0.0-mvp/database/migrations/002_staged_rollout.up.sql</code>), already real-DB validated. The 1.0.2 version slot is free; delta-updates stays at 1.0.3 .
Superseded-Details	Folds addition-#3’s <code>ROLLBACK_DESIGN.md</code> into the canonical numbering. Numbering is LOCKED per synthesis §10 K8: staged-rollout owns 1.0.1 , so rollback lands at 1.0.2 (the source doc self-numbered 1.0.1; re-based here). Scope is the operator/server-triggered + user-initiated + multi-version superset of rollback. Automatic A/B boot-failure rollback is already an MVP guarantee (AOSP boot-control HAL) and is not new work here — this phase
Status summary	

Field	Value
	adds the human/operator-driven paths on top of it.
Issues	Source doc carries the addition-#3 stack drift (RSA/mTLS/rollouts+updates vocabulary, /api/v1/... paths, “automatic rollback” framed as new). All re-based onto locked decisions below. Multi-version / true-downgrade is explicitly out of A/B’s single-previous-slot reach and needs its own sub-design. Scope boundary with 1.0.1 end-user-rollback bullet must be settled (see §scope_boundary_vs_1_0_1).
Issues summary	Accept the rollback mechanics; reject the divergent stack/vocabulary; do not double-count automatic boot-failure rollback.
Fixed	initial research-routed outline
Fixed summary	Captured the three rollback trigger types, the merge-window constraint, the data-safety invariant, and the rollback API surface — all reconciled to releases/deployments + ed25519 + JWT + Gin.
Continuation	Expand each section into full specs under this directory (rollback engine spec, migration 003_* for rollback history, device-side rollback-detection spec, settings-UX spec), each adversarially reviewed and validated against live Postgres + a real Orange Pi 5 Max, then exported.

table_of_contents

- [§1. scope](#)
- [§2. source_research](#)
- [§3. reconciliation_to_locked_decisions](#)
- [§4. rollback_trigger_types](#)
- [§5. merge_window_and_data_safety_invariants](#)
- [§6. api_and_data_model_surface](#)
- [§7. scope_boundary_vs_1_0_1](#)
- [§8. what_must_be_specced_before_this_phase_starts](#)
- [§9. anti_bluff_unverified_register](#)

§1. scope

Operator- and user-driven reversion of a fleet (or single device) to its previous known-good build, plus the groundwork for true multi-version downgrade. The Helix control plane (artifact intake, signing/verify, deployments, telemetry) is reused; this phase adds rollback orchestration on the server and rollback-detection/initiation on the device. The **automatic boot-failure A/B fallback** is an existing MVP property and is treated here only as the substrate the operator-initiated paths sit on, not as new deliverable code.

§2. source_research

Source: additions/initial_research_03/helix_ota_big/1.0.1-rollback/docs/ROLLBACK_DESIGN.md (HELOTA-ROLLBACK-001, self-versioned 1.0.1, re-based to 1.0.2 here). Cited sections, summarized (NOT copied):

- **§1 Rollback Overview** — motivation (2–5% of OTAs regress per its claim, see UNVERIFIED), the three trigger types, and the rollback-vs-downgrade distinction (§1.3 table): rollback = instant A/B slot swap to the immediate previous version; downgrade = full OTA cycle to an arbitrary older version. 1.0.2 owns rollback; true downgrade is a separate sub-design.
- **§2 Android A/B Rollback Mechanism** — boot-control HAL (`setActiveBootSlot` / `getActiveBootSlot` / `markBootSuccessful`), the 3-retry auto-fallback counter, and the **Virtual A/B merge boundary** (§2.4): slot-swap rollback is only possible *before* the COW merge completes; after merge the previous slot is gone. dm-verity re-verification of the rolled-back slot (§2.5).
- **§3 Server-Side Rollback** — single-device, fleet/rollout, and history endpoints; trigger conditions (failure-rate threshold, boot-loop detection, health-check failure pattern, anomaly rules); instant vs gradual (wave) execution; append-only audit table.
- **§4 Client-Side Rollback** — boot-failure detection on next boot, the post-boot `HealthCheckService` battery (§4.3) that decides rollback before `markBootSuccessful()`, user-initiated rollback from Settings (§4.4) with a default 7-day window, and rollback state reporting via telemetry (§4.5).
- **§5 Version History Management** — A/B inherently supports exactly **one** previous version; multi-version rollback would need extra slots (unsupported on RK3588), /data-stored images, or server-side downgrade — all deferred (§5.1).
- **§6 Data Safety** — the cardinal invariant: `userdata` is never touched by a slot swap (§6.1); forward-compatible app-data migration patterns (§6.3); config-snapshot/migrate on rollback (§6.4).
- **§7 Rollback API Specification, §8 Rollback Testing** (RB-001..RB-007 + power-failure matrix), **§9 Go reference engine, §10 Kotlin reference client** — design seeds, re-based per §3 below.

§3. reconciliation_to_locked_decisions

Source-doc element	As written	Reconciled (LOCKED)
Router / API style	/api/v1/..., implied generic	Gin , canonical Helix path scheme (no /api/v1 prefix assumption);

Source-doc element	As written	Reconciled (LOCKED)
Signing	inherits addition-#3 RSA elsewhere	REST primary, HTTP/3→HTTP/2, Brotli ed25519 + SHA-256 (synthesis K2); rollback authorization tokens are JWT-bearer signed, artifacts stay ed25519
Device transport auth	Bearer JWT (already correct in source §7)	JWT bearer (synthesis K3) — kept
Vocabulary	rollouts, updates, rollback_audit_log, device_group	canonical releases + deployments ; rollout correlation rides the 1.0.1 deployments/rollouts model; rollback history table named per database/schema.md convention
Module path	github.com/helix-ota/..., vasic-digital/* imports	github.com/HelixDevelopment/helix_ota (synthesis K4); reuse verified catalogue bricks (eventbus, cache, Storage, observability, ratelimiter), not invented vasic-digital/concurrency
Automatic boot-failure rollback	framed as a 1.0.x deliverable	already MVP (AOSP boot-control HAL + AVB); this phase is the <i>operator/user-initiated superset</i> , plus the report-after-the-fact detection path
Coverage	85% (addition-#3 default)	≥90% floor on safety-critical paths (slot-swap authorization, merge-window check, downgrade-index interplay) per §1/C8

§4. rollback_trigger_types

Three triggers, all converging on the same A/B slot-swap substrate (source §1.2, §4):

1. **Automatic (boot failure)** — hardware/bootloader path, **already MVP**. New here: the device-side RollbackDetectionService that, on next successful boot, notices it is back on the pre-update slot and reports a ROLLBACK telemetry

- event (trigger_type automatic); the server updates current_version/slot and writes an audit record.
2. **Server/operator-triggered** — admin/operator invokes single-device or fleet rollback; server classifies devices into pre-merge (slot-swappable), post-merge (needs downgrade artifact), and offline (queued), then injects a rollback directive into the next update-check response. Instant or gradual (wave) execution. This is the primary new control-plane work.
 3. **User-initiated** — on-device Settings entry, available only while a bootable previous slot exists, the Virtual A/B merge is still pending, and the rollback window (default 7 days) has not expired.

§5. merge_window_and_data_safety_invariants

- **Merge-window constraint (LOAD-BEARING):** slot-swap rollback is possible **only before the Virtual A/B COW merge completes** (source §2.4). The device must therefore delay markBootSuccessful() until the post-boot health window passes — this is the same health-gated mechanism 1.0.1 introduces, so 1.0.2 *depends on* 1.0.1's health-confirmation window rather than re-inventing it.
- **Data-safety invariant:** userdata is never modified by a rollback; slot swap changes only boot-control metadata (source §6.1). This must be pinned by a pre/post integrity test.
- **Anti-downgrade interplay:** authorized rollback must not be blocked by, nor become an attack against, the AVB rollback-index / anti-downgrade invariant (G1 in synthesis §8; 1.0.1 §5). The rollback authorization must be distinguishable from a malicious downgrade offer.

§6. api_and_data_model_surface

Design seeds from source §7/§9, to be re-based onto canonical paths and database/schema.md:

- Endpoints: single-device rollback, deployment/rollout rollback (instant|gradual waves, dry-run), and rollback-history read — all JWT-gated by admin/operator role, rate-limited to prevent fleet-wide cascade. Viewer role read-only.
- Telemetry: a ROLLBACK event type carrying trigger_type, from/to version, from/to slot, and health-check results — extends ota-telemetry-schema (single-source the enum, do not redefine).
- Data model: a rollback-history / rollback-audit table (append-only, immutable) + device previous_version / slot_suffix / merge_completed fields. Lands in migration 003_* (1.0.1 owns 002_*).

§7. scope_boundary_vs_1_0_1

The 1.0.1 outline currently lists **end-user rollback** as one of its bullets (1.0.1 §5). With K8 locking rollback at 1.0.2, that bullet must be reconciled: either (a) 1.0.1 retains only the *enablers* (health-gated merge delay, A/B slot-pinning hooks, anti-downgrade preservation) and 1.0.2 owns all operator/user rollback UX + orchestration, or (b) end-user rollback formally moves to 1.0.2. **Recommended: (a)**

— 1.0.1 ships the substrate, 1.0.2 ships the rollback product. This boundary **MUST** be finalized before either phase is specced to avoid duplicate ownership. (UNVERIFIED until ratified.)

§8. what _must_ be _specced_ _before_ _this_ _phase_ _starts

1. **Settle the 1.0.1↔1.0.2 boundary** (§7) — single source of ownership for end-user rollback.
2. **Rollback engine spec** — server orchestration (classify → directive → monitor), reusing eventbus/cache; instant vs gradual wave logic; idempotency + halt-wins alignment with the 1.0.1 rollout engine.
3. **Migration 003_*** — rollback history/audit table + device slot/merge/previous-version fields; up/down validated against live Postgres.
4. **Device-side rollback-detection + report spec** — slot comparison on boot, telemetry event, no `markBootSuccessful()` on the failed slot.
5. **Health-window dependency** — confirm 1.0.1 delivers the merge-delay health window 1.0.2 relies on; document the dependency edge.
6. **Anti-downgrade / AVB rollback-index interplay spec** — authorized-rollback vs malicious-downgrade distinction (ties to G1).
7. **User-initiated Settings UX spec** — availability predicates (bootable prior slot, merge pending, window unexpired) and confirmation flow.
8. **Multi-version / true-downgrade sub-design** — explicitly out of single-slot A/B; route to its own design (downgrade artifact + forward migration + security-patch-level handling) before promising it.
9. **Test plan** — RB-001..RB-007 + power-failure matrix re-based to canonical stack; four-layer coverage with $\geq 90\%$ on the slot-swap-authorization and merge-window paths; real Orange Pi 5 Max validation.

§9. anti_bluff_unverified_register

Per Constitution §11.4.6 (no-guessing) / §7.1 (no-bluff) — **MUST NOT** propagate as fact:

- **“2–5% of OTA updates regress”** (source §1.1) — UNVERIFIED industry claim; do not cite as a Helix metric.
- **3-retry boot counter, 60s health window, 7-day rollback window, 90s recovery time** — source defaults, UNVERIFIED on RK3588 / Android 15; confirm on real hardware before binding.
- **Virtual A/B merge behavior under mid-merge rollback** (source §2.4) — UNVERIFIED on the Orange Pi 5 Max `snapuserd`/bootloader; hardware-gated spike required.
- **dm-verity / verified-boot-state transitions on rollback + key rotation** (source §2.5) — UNVERIFIED on RK3588.
- **Failure-rate / boot-loop / anomaly thresholds** (source §3.2: 5%, 10 min, $>300\%$ crash rate) — UNVERIFIED; set from measured MVP/1.0.1 data, do not assert.
- Source-doc **Go/Kotlin reference code** imports invented modules (`vasic-digital/concurrency`) and rollouts/updates vocabulary — reference only; not canonical until re-based.
- All cited **HelixConstitution §11.4.x** clause numbers remain UNVERIFIED except the six confirmed in `tests/test_strategy.md` §13.